PINAKI IT CONSULTANT
PRIVATE LIMITED

FREE SAMPLE NOTES

SQL BASICS

A Beginner's Guide to SQL and Database Management

PINAKI IT CONSULTANT PRIVATE LIMITED

Welcome to your free sample of our SQL notes. These pages are written for students who have never worked with a database. Every topic starts with simple theory, then shows the syntax, a realistic query on one consistent student database, its exact result, and a clear explanation of how it works.

WHAT YOU WILL LEARN

- Databases, tables and keys
- Basic syntax and command categories
- Creating databases and tables
- INSERT, UPDATE, DELETE and ALTER
- SELECT, filtering and sorting
- Aggregate functions and GROUP BY
- JOINs across tables
- Constraints and relationships
- Subqueries and CASE expressions
- A complete mini project

```sql
-- Your first useful query
SELECT Name, Marks
FROM Students
ORDER BY Marks DESC
LIMIT 3;
```

RESULT

Name	Marks
Diya Patel	92.00
Meera Nair	88.00
Aarav Sharma	85.50

Learn more at **www.pinakiithub.com**

1 Introduction to SQL and Databases

What is SQL?

SQL stands for **Structured Query Language**. It is the standard language used to store, retrieve, update and manage data in **relational databases**. A **query** is simply a question or instruction sent to the database, such as "show me every student who scored above 75".

What is a database?

A **database** is an organised collection of data stored electronically so that it can be searched and updated quickly. Banks, shops, schools and hospitals use databases because paper files and ordinary spreadsheets cannot keep large amounts of data accurate while many people use it at once.

What is a table?

A relational database stores data in **tables**. A table is a grid of **rows** and **columns**, much like a sheet in a spreadsheet. Different tables can be linked to each other, which is why the database is called "relational".

Term	Meaning	Example
Row (record)	One complete entry in a table	All details of one student
Column (field)	One kind of information stored for every row	Marks
Data type	The kind of value a column can hold	INT, VARCHAR, DATE
Primary key	A column that uniquely identifies each row. It cannot repeat and cannot be empty (NULL)	StudentID

A sample Students table (this table is used throughout these notes). Here `StudentID` is the primary key.

StudentID	Name	Age	Marks
1	Aarav Sharma	20	85.50
2	Diya Patel	21	92.00
3	Rohan Mehta	22	72.00
4	Sneha Iyer	20	74.50

Why is SQL important?

Area	How SQL is used
Data analytics	Analysts pull and summarise data from databases before charting or modelling it
Banking	Storing accounts and transactions and checking balances
E-commerce	Managing products, orders, payments and stock levels
Business reporting	Producing sales, finance and performance reports
Customer management	Keeping customer details and purchase history together
Web applications	Websites and apps read and save their data with SQL

SQL vs. MySQL

SQL is a language; MySQL is software. SQL is the set of commands you write. **MySQL** is a **relational database management system (RDBMS)**, a program that stores the data and understands SQL commands. Other popular systems include **PostgreSQL**, **Microsoft SQL Server** and **SQLite** (a lightweight database that lives in a single file). All of them use SQL, with small differences.

2 SQL Environment and Basic Syntax

An **RDBMS** is the software that stores your tables, runs your queries and controls who may access the data. These notes use **MySQL 8.0** as the example system. A **SQL query** (also called a statement) is one instruction written in SQL.

How a query is executed

1. You type the query in a client tool such as **MySQL Workbench**.
2. The client sends it to the database server, which checks the syntax.
3. The server runs the query on the tables and sends back the **result set** (or a success message).

PRO TIP

Optional quick setup: download the free MySQL Community Server and MySQL Workbench from mysql.com, connect to your local server, open a new query tab, type a query and press **Ctrl + Enter** to run it. Any other MySQL client works equally well.

Basic syntax rules

- **Keywords** such as `SELECT` and `FROM` are not case-sensitive, but writing them in UPPERCASE makes queries easier to read.
- A **semicolon** `;` marks the end of a statement.
- **Comments** are ignored by the database: `-- single line` or `/* multiple lines */`.
- Use table and column names consistently. On some systems (for example MySQL on Linux) table names are case-sensitive.

SQL

```
SELECT 'Hello, SQL!';
```

RESULT

Hello, SQL!
Hello, SQL!

Part of the query	Meaning
`SELECT`	Keyword that asks the database to return values.
`'Hello, SQL!'`	A **string literal**: text written inside single quotation marks.
`;`	Ends the statement.

Readable formatting

The two queries below do exactly the same thing. Put each clause on its own line and indent continuation lines.

HARD TO READ

```
SELECT Name,Marks FROM Students
WHERE Marks>=75 ORDER BY Marks DESC;
```

EASY TO READ

```
-- Students with 75 or more marks, highest first
SELECT Name, Marks
FROM Students
WHERE Marks >= 75
ORDER BY Marks DESC;
```

REMEMBER

SQL syntax and some functions vary between database systems. These notes use MySQL 8.0 syntax and point out important differences as they appear.

3 SQL Command Categories

SQL commands are grouped by what they do. Knowing the groups helps you understand which command to reach for. This page is an overview; each command is explained with examples on the pages that follow.

Category	Purpose	Common commands	Short use case
DDL **Data Definition Language**	Creates and changes the **structure** of the database	CREATE, ALTER, DROP	Create a Students table
DML **Data Manipulation Language**	Adds, changes and removes the **data** inside tables	INSERT, UPDATE, DELETE	Add a new student record
DQL **Data Query Language**	**Reads** data without changing it	SELECT	List students who scored above 75
DCL **Data Control Language**	Controls **who is allowed** to do what	GRANT, REVOKE	Let an analyst read but not change student data
TCL **Transaction Control Language**	Saves or cancels a group of changes made together	COMMIT, ROLLBACK	Save a bank transfer only if both steps succeed

A library analogy

Imagine a library. Each SQL category matches a job in it:

Library job	SQL category
Building the shelves and labelling sections	DDL: define the structure
Adding, updating and removing books on the shelves	DML: change the data
Searching the catalogue for a book	DQL: read the data
Deciding who gets a library card and what it allows	DCL: control access
Confirming a whole batch of returned books at once, or cancelling it	TCL: save or undo a group of changes

Where you will see them in these notes

- **DDL** appears on pages 5, 6, 10 and 11 (`CREATE`, `ALTER`) and **DML** on page 6 (`INSERT`, `UPDATE`, `DELETE`).
- **DQL** (`SELECT`) is the focus of pages 7 to 13, because reading and analysing data is the most common SQL task.
- **DCL** and **TCL** are introduced here only; they are covered in depth in the complete notes.

> **REMEMBER**
> Different books and database systems classify a few commands slightly differently. For example, some materials place `SELECT` under DML. The overall idea stays the same.

4 Databases, Tables and Data Types

Before storing data, we create a **database** (a container for related tables) and then create **tables** inside it. A **data type** tells the database what kind of value each column accepts.

CREATE DATABASE and USE

SYNTAX

```
CREATE DATABASE database_name;

USE database_name;
```

EXAMPLE

```
CREATE DATABASE
StudentDB;

USE StudentDB;
```

RESULT

```
Query OK, 1 row affected
Database changed
```

`CREATE DATABASE` makes the container and `USE` selects it, so later commands apply to `StudentDB`. `USE` is specific to MySQL and some other systems.

Basic data types

Type	Stores	Example value
INT	Whole numbers	20
VARCHAR(n)	Text of up to n characters	'Python'
DATE	A calendar date (YYYY-MM-DD)	'2025-06-30'
DECIMAL(p, s)	Exact decimals: p digits in total, s after the point	85.50 in DECIMAL(5,2)

NULL means “no value” or “unknown”. It is not the same as 0 or an empty text. You test for it with `IS NULL`.

CREATE TABLE

SYNTAX

```
CREATE TABLE table_name (
    column1 data_type,
    column2 data_type
);
```

EXAMPLE

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT,
    Course VARCHAR(50),
    Marks DECIMAL(5,2)
);
```

RESULT

```
Query OK, 0 rows affected
```

Column	Data type	What it stores
StudentID	INT, PRIMARY KEY	Unique number for each student
Name	VARCHAR(50)	Student’s full name (up to 50 characters)
Age	INT	Age in years
Course	VARCHAR(50)	Course the student is following
Marks	DECIMAL(5,2)	Marks out of 100, such as 85.50 (up to 999.99)

Line by line: `CREATE TABLE Students (` starts the table; each line inside the brackets defines one column (a name, then a data type) separated by commas, with no comma after the last one; `);` ends the statement. A **primary key** makes `StudentID` unique and never empty, so every student can be found precisely.

PRO TIP

Choose data types carefully: `INT` for counts, `DECIMAL` for exact values such as marks or money, `VARCHAR` for text. Good types prevent wrong data and save space.

5 INSERT, UPDATE, DELETE and ALTER

INSERT: add new rows

`INSERT INTO` adds one or more records. List the columns, then give the matching values in the same order. Text goes in single quotes; numbers do not.

SYNTAX

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

SQL

```
INSERT INTO Students (StudentID, Name, Age, Course, Marks)
VALUES (1, 'Aarav Sharma', 20, 'Python', 85.50);
```

RESULT

```
Query OK, 1 row affected
```

SQL

```
INSERT INTO Students (StudentID, Name, Age, Course, Marks)
VALUES
    (2, 'Diya Patel', 21, 'SQL', 92.00),
    (3, 'Rohan Mehta', 22, 'Python', 68.00),
    (4, 'Sneha Iyer', 20, 'Excel', 74.50),
    (5, 'Kabir Singh', 23, 'SQL', 55.00),
    (6, 'Meera Nair', 21, 'Power BI', 88.00),
    (7, 'Arjun Verma', 22, 'Python', 79.00),
    (8, 'Isha Gupta', 20, 'Excel', 61.00),
    (9, 'Neha Kapoor', 24, 'Excel', 70.00);
```

RESULT

```
Query OK, 8 rows affected
```

The second statement inserts eight rows at once. Neha Kapoor (ID 9) is added so that we can delete her below.

UPDATE: change existing rows

`UPDATE` changes values in the rows that match the `WHERE` condition. Here Rohan's marks (StudentID 3) are corrected from 68.00 to 72.00.

SYNTAX

```
UPDATE table_name
SET column = value
WHERE condition;
```

EXAMPLE

```
UPDATE Students
SET Marks = 72.00
WHERE StudentID = 3;
```

RESULT

```
Query OK, 1 row affected
```

DELETE: remove rows

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

EXAMPLE

```
DELETE FROM Students
WHERE StudentID = 9;
```

RESULT

```
Query OK, 1 row affected
```

ALTER TABLE: change the structure

`ALTER TABLE` changes the table design itself. We add an `Email` column (it is `NULL` for every row at first) and then store one email address.

SYNTAX

```
ALTER TABLE table_name
ADD column_name
data_type;
```

EXAMPLE

```
ALTER TABLE Students
ADD Email VARCHAR(100);

UPDATE Students
SET Email = 'aarav@example.com'
WHERE StudentID = 1;
```

RESULT

```
Query OK, 0 rows affected

Query OK, 1 row affected
```

COMMON MISTAKES

Never run UPDATE or DELETE without a suitable WHERE condition: it would affect every row. Check the rows first with a `SELECT`. The table now holds **eight students** (IDs 1 to 8), the dataset for the rest of these notes.

6 SELECT Statement and Filtering Data

`SELECT` reads data from a table and is the most important SQL command. `*` means "all columns"; `WHERE` keeps only the rows that satisfy a condition; `AS` gives a column a temporary, friendlier name (an **alias**); `DISTINCT` removes duplicates.

SYNTAX

```
SELECT column1, column2 (or *)
FROM table_name
WHERE condition;
```

Specific columns

```sql
SELECT Name, Marks
FROM Students;
```

RESULT

Name	Marks
Aarav Sharma	85.50
Diya Patel	92.00
Rohan Mehta	72.00
Sneha Iyer	74.50
Kabir Singh	55.00
Meera Nair	88.00
Arjun Verma	79.00
Isha Gupta	61.00

BETWEEN includes both end values.

```sql
SELECT Name, Marks
FROM Students
WHERE Marks BETWEEN 60 AND 90;
```

RESULT

Name	Marks
Aarav Sharma	85.50
Rohan Mehta	72.00
Sneha Iyer	74.50
Meera Nair	88.00
Arjun Verma	79.00
Isha Gupta	61.00

Alias with AS and a **WHERE** filter on text.

```sql
SELECT Name AS StudentName,
       Marks AS Score
FROM Students
WHERE Course = 'Python';
```

RESULT

StudentName	Score
Aarav Sharma	85.50
Rohan Mehta	72.00
Arjun Verma	79.00

DISTINCT lists each course once.

```sql
SELECT DISTINCT Course
FROM Students;
```

RESULT

Course
Python
SQL
Excel
Power BI

IS NULL finds missing values; **AND** needs both conditions.

```sql
SELECT Name
FROM Students
WHERE Email IS NULL
  AND Marks >= 85;
```

RESULT

Name
Diya Patel
Meera Nair

All columns, filtered: students with 75 or more marks. The `>=` comparison operator means "greater than or equal to".

```sql
SELECT *
FROM Students
WHERE Marks >= 75;
```

RESULT

StudentID	Name	Age	Course	Marks	Email
1	Aarav Sharma	20	Python	85.50	aarav@example.com
2	Diya Patel	21	SQL	92.00	*NULL*
6	Meera Nair	21	Power BI	88.00	*NULL*
7	Arjun Verma	22	Python	79.00	*NULL*

Note: without `ORDER BY`, SQL does not guarantee row order; the examples show the natural order of the table.

7 SQL Operators, Sorting and Limiting Results

Operators build the conditions used in `WHERE` and the calculations used in `SELECT`. This quick reference reinforces the ones you met on the previous page.

Category	Operators	Example	Meaning
Arithmetic	`+ - * / %`	`Marks + 5`	Calculates a new value for each row
Comparison	`= != > < >= <=`	`Marks >= 75`	Compares two values
Logical	`AND OR NOT`	`Course = 'SQL' AND Marks > 80`	Combines conditions
Pattern	`LIKE`	`Name LIKE 'A%'`	% = any characters, _ = one character
List	`IN`	`Course IN ('Python', 'SQL')`	Matches any value in a list
Range	`BETWEEN`	`Marks BETWEEN 60 AND 90`	Between two values, both included
Missing value	`IS NULL`	`Email IS NULL`	Value is missing (IS NOT NULL is the opposite)

ORDER BY and LIMIT

`ORDER BY` sorts the result: `ASC` means ascending (smallest first, the default) and `DESC` descending (largest first). `LIMIT` keeps only the first rows. Syntax: `ORDER BY column ASC | DESC` and `LIMIT number`.

SQL

```sql
SELECT *
FROM Students
ORDER BY Marks DESC;
```

RESULT

StudentID	Name	Age	Course	Marks	Email
2	Diya Patel	21	SQL	92.00	*NULL*
6	Meera Nair	21	Power BI	88.00	*NULL*
1	Aarav Sharma	20	Python	85.50	aarav@example.com
7	Arjun Verma	22	Python	79.00	*NULL*
4	Sneha Iyer	20	Excel	74.50	*NULL*
3	Rohan Mehta	22	Python	72.00	*NULL*
8	Isha Gupta	20	Excel	61.00	*NULL*
5	Kabir Singh	23	SQL	55.00	*NULL*

All eight students are sorted from the highest marks down. Combine sorting with `LIMIT` for a “top N” report:

ORDER BY + LIMIT: the top 3 students.

SQL

```sql
SELECT Name, Marks
FROM Students
ORDER BY Marks DESC
LIMIT 3;
```

RESULT

Name	Marks
Diya Patel	92.00
Meera Nair	88.00
Aarav Sharma	85.50

Arithmetic + LIKE: 5 bonus marks for names starting with A.

SQL

```sql
SELECT Name, Marks + 5 AS BonusMarks
FROM Students
WHERE Name LIKE 'A%';
```

RESULT

Name	BonusMarks
Aarav Sharma	90.50
Arjun Verma	84.00

`LIMIT` works in MySQL and several other systems; SQL Server uses `TOP 3`, and Oracle uses `FETCH FIRST 3 ROWS ONLY`.

8 Aggregate Functions and GROUP BY

An **aggregate function** looks at many rows and returns **one summary value**. Summaries like these power business reports and analytics dashboards.

Function	Returns	Example
`COUNT()`	Number of rows	`COUNT(*)`
`SUM()`	Total of a numeric column	`SUM(Marks)`
`AVG()`	Average value	`AVG(Marks)`
`MIN()`	Smallest value	`MIN(Marks)`
`MAX()`	Largest value	`MAX(Marks)`

`COUNT(*)` counts all rows.

```sql
SELECT COUNT(*) AS TotalStudents
FROM Students;
```

RESULT

TotalStudents
8

AVG() with ROUND() for two decimals.

```sql
SELECT ROUND(AVG(Marks), 2) AS
AverageMarks
FROM Students;
```

RESULT

AverageMarks
75.88

```sql
SELECT SUM(Marks) AS TotalMarks,
       MIN(Marks) AS LowestMarks,
       MAX(Marks) AS HighestMarks
FROM Students;
```

RESULT

TotalMarks	LowestMarks	HighestMarks
607.00	55.00	92.00

NULL values: aggregate functions generally ignore `NULL`, except `COUNT(*)`, which counts rows. Only one student has an email so far:

```sql
SELECT COUNT(*) AS AllRows,
       COUNT(Email) AS WithEmail
FROM Students;
```

RESULT

AllRows	WithEmail
8	1

GROUP BY: one summary per group

Grouping splits the rows into groups that share the same value, then applies the aggregate function to each group separately. Every column in `SELECT` that is not inside an aggregate function must appear in `GROUP BY`.

```
SYNTAX
SELECT column, AGGREGATE_FUNCTION(column)
FROM table_name
GROUP BY column;
```

```sql
SELECT Course,
       COUNT(*) AS Students,
       ROUND(AVG(Marks), 2) AS AverageMarks
FROM Students
GROUP BY Course
ORDER BY Course;
```

RESULT

Course	Students	AverageMarks
Excel	2	67.75
Power BI	1	88.00
Python	3	78.83
SQL	2	73.50

Each course becomes one group: Python has 3 students averaging 78.83, and so on. The `ORDER BY` simply keeps the groups in alphabetical order. **Practical use:** sales per region, orders per month or average salary per department.

9 SQL JOINs

Related information is kept in separate tables to avoid repeating it. A **JOIN** combines rows from two tables using a **matching column**. We add a `Courses` table and a `CourseID` column in Students to link them.

SQL: CREATE AND FILL COURSES

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50)
);

INSERT INTO Courses VALUES
    (1, 'Python'), (2, 'SQL'), (3, 'Excel'),
    (4, 'Power BI'), (5, 'Data Visualization');
```

RESULT OF SELECT * FROM Courses

CourseID	CourseName
1	Python
2	SQL
3	Excel
4	Power BI
5	Data Visualization

SQL

```
ALTER TABLE Students ADD CourseID INT;
UPDATE Students SET CourseID = 1 WHERE Course = 'Python';
UPDATE Students SET CourseID = 2 WHERE Course = 'SQL';
UPDATE Students SET CourseID = 3 WHERE Course = 'Excel';
UPDATE Students SET CourseID = 4 WHERE Course = 'Power BI';
```

RESULT

```
Query OK (ALTER)
3, 2, 2 and 1 rows
affected by the
UPDATEs
```

INNER JOIN returns only rows that match in both tables. **LEFT JOIN** keeps all rows of the left table (the one after `FROM`) plus any matches, and **RIGHT JOIN** keeps all rows of the right table; where there is no match, the other side's columns are `NULL`. **FULL OUTER JOIN** (all rows of both) is not supported directly in MySQL. In the queries below `s` and `c` are table aliases and `ON` names the matching columns.

INNER JOIN: each student with their course.

SQL

```
SELECT s.Name, c.CourseName
FROM Students s
INNER JOIN Courses c
    ON s.CourseID = c.CourseID
ORDER BY s.StudentID;
```

RESULT

Name	CourseName
Aarav Sharma	Python
Diya Patel	SQL
Rohan Mehta	Python
Sneha Iyer	Excel
Kabir Singh	SQL
Meera Nair	Power BI
Arjun Verma	Python
Isha Gupta	Excel

LEFT JOIN from Courses: all courses are kept.

SQL

```
SELECT c.CourseName, s.Name
FROM Courses c
LEFT JOIN Students s
    ON c.CourseID = s.CourseID
ORDER BY c.CourseID, s.StudentID;
```

RESULT

CourseName	Name
Python	Aarav Sharma
Python	Rohan Mehta
Python	Arjun Verma
SQL	Diya Patel
SQL	Kabir Singh
Excel	Sneha Iyer
Excel	Isha Gupta
Power BI	Meera Nair
Data Visualization	*NULL*

Data Visualization has no students, so its `Name` is `NULL`. Swapping the tables gives a `RIGHT JOIN`:

SQL

```
SELECT s.Name, c.CourseName
FROM Students s
RIGHT JOIN Courses c ON s.CourseID = c.CourseID
ORDER BY c.CourseID, s.StudentID;
```

RESULT

```
9 rows: the same rows as the
LEFT JOIN, with Name first and
CourseName second.
```

10 SQL Constraints and Keys

A **constraint** is a rule the database enforces automatically, so wrong or inconsistent data is rejected. Constraints protect the quality of your data.

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row; no duplicates, no NULL	`StudentID INT PRIMARY KEY`
FOREIGN KEY	Links a column to the primary key of another table	`FOREIGN KEY (CourseID) REFERENCES Courses (CourseID)`
NOT NULL	The column must always have a value	`Name VARCHAR(50) NOT NULL`
UNIQUE	No two rows may have the same value	`Email VARCHAR(100) UNIQUE`
DEFAULT	Supplies a value when none is given	`Status VARCHAR(20) DEFAULT 'Active'`
CHECK	The value must satisfy a condition	`CHECK (Marks BETWEEN 0 AND 100)`

CHECK note: exact behaviour can depend on the database system and version. MySQL enforces `CHECK` from version 8.0.16; earlier versions accept it but ignore it.

Primary keys, foreign keys and referential integrity

`Courses.CourseID` is the **primary key** of Courses. `Students.CourseID` is a **foreign key**: it must always match an existing course. This rule is called **referential integrity**: a student can never point to a course that does not exist, and a course that still has students cannot simply be deleted.

SQL
```
ALTER TABLE Students
ADD CONSTRAINT fk_students_course
FOREIGN KEY (CourseID)
REFERENCES Courses (CourseID);
```

RESULT
```
Query OK, 8 rows affected
Records: 8  Duplicates: 0
```

Now the database rejects an invalid course. There is no course 99:

SQL
```
INSERT INTO Students
    (StudentID, Name, Age, Course, Marks, CourseID)
VALUES (10, 'Test User', 21, 'Java', 60.00, 99);
```

RESULT
```
ERROR 1452 (23000): Cannot
add or
update a child row: a
foreign key
constraint fails
```

Several constraints together

SQL
```
CREATE TABLE StudentsDemo (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Status VARCHAR(20) DEFAULT 'Active',
    Marks DECIMAL(5,2) CHECK (Marks BETWEEN 0 AND 100)
);
```

RESULT
```
Query OK, 0 rows
affected
```

`NOT NULL` forces a name; `UNIQUE` stops two students sharing an email; `DEFAULT` sets the status to Active when none is given; `CHECK` rejects marks outside 0 to 100. `StudentsDemo` is only an illustration of the syntax; the notes continue with the `Students` table.

11 Subqueries, CASE Expressions and Practical Queries

Subquery: a query inside a query

A **subquery** is a `SELECT` written inside another query, usually in brackets. The **inner query runs first** and its result is used by the **outer query**. Below, the inner query finds the average marks (75.875) and the outer query keeps students above it.

Students above the class average

```sql
SELECT Name, Marks
FROM Students
WHERE Marks > (
    SELECT AVG(Marks)
    FROM Students
);
```

RESULT

Name	Marks
Aarav Sharma	85.50
Diya Patel	92.00
Meera Nair	88.00
Arjun Verma	79.00

Practical: the highest-scoring student.

```sql
SELECT Name, Marks
FROM Students
WHERE Marks = (
    SELECT MAX(Marks)
    FROM Students
);
```

RESULT

Name	Marks
Diya Patel	92.00

Practical: how many students need support (marks below 70)?

```sql
SELECT COUNT(*) AS
NeedsImprovement
FROM Students
WHERE Marks < 70;
```

RESULT

NeedsImprovement
2

CASE: labelling rows

`CASE` works like if-then-else inside a query: conditions are checked in order and the first true one wins.

Marks	Category
85 or more	Excellent
70 to below 85	Good
Below 70	Needs Improvement

```sql
SELECT Name, Marks,
  CASE
    WHEN Marks >= 85 THEN 'Excellent'
    WHEN Marks >= 70 THEN 'Good'
    ELSE 'Needs Improvement'
  END AS Category
FROM Students;
```

RESULT

Name	Marks	Category
Aarav Sharma	85.50	Excellent
Diya Patel	92.00	Excellent
Rohan Mehta	72.00	Good
Sneha Iyer	74.50	Good
Kabir Singh	55.00	Needs Improvement
Meera Nair	88.00	Excellent
Arjun Verma	79.00	Good
Isha Gupta	61.00	Needs Improvement

Practical: which course has the best average marks?

```sql
SELECT Course, ROUND(AVG(Marks), 2)
       AS AverageMarks
FROM Students
GROUP BY Course
ORDER BY AverageMarks DESC
LIMIT 1;
```

RESULT

Course	AverageMarks
Power BI	88.00

Practical use: subqueries answer “compared with the overall figure” questions, `CASE` turns raw numbers into report categories, and `GROUP BY` with `ORDER BY` and `LIMIT` produces league tables.

12 Mini Project

STUDENT PERFORMANCE MANAGEMENT SYSTEM

Objective: use SQL to store student records and analyse their performance. The project uses the `Students` table built in these notes (`StudentID`, `Name`, `Age`, `Course`, `Marks`, `Email`) with the eight sample records shown in query 1. Every result below is produced by the query next to it.

1. Retrieve all students

```sql
SELECT StudentID, Name, Course, Marks
FROM Students;
```

RESULT

StudentID	Name	Course	Marks
1	Aarav Sharma	Python	85.50
2	Diya Patel	SQL	92.00
3	Rohan Mehta	Python	72.00
4	Sneha Iyer	Excel	74.50
5	Kabir Singh	SQL	55.00
6	Meera Nair	Power BI	88.00
7	Arjun Verma	Python	79.00
8	Isha Gupta	Excel	61.00

2. Sort students by marks

```sql
SELECT Name, Marks
FROM Students
ORDER BY Marks DESC;
```

RESULT

Name	Marks
Diya Patel	92.00
Meera Nair	88.00
Aarav Sharma	85.50
Arjun Verma	79.00
Sneha Iyer	74.50
Rohan Mehta	72.00
Isha Gupta	61.00
Kabir Singh	55.00

3. Students who scored above 80

```sql
SELECT Name, Marks
FROM Students
WHERE Marks > 80;
```

RESULT

Name	Marks
Aarav Sharma	85.50
Diya Patel	92.00
Meera Nair	88.00

4. Average marks

```sql
SELECT ROUND(AVG(Marks), 2) AS
AverageMarks
FROM Students;
```

RESULT

AverageMarks
75.88

5. Count students by course

```sql
SELECT Course, COUNT(*) AS Students
FROM Students
GROUP BY Course
ORDER BY Course;
```

RESULT

Course	Students
Excel	2
Power BI	1
Python	3
SQL	2

6. Top three students

```sql
SELECT Name, Course, Marks
FROM Students
ORDER BY Marks DESC
LIMIT 3;
```

RESULT

Name	Course	Marks
Diya Patel	SQL	92.00
Meera Nair	Power BI	88.00
Aarav Sharma	Python	85.50

How it works: query 1 reads the records, 2 uses `ORDER BY ... DESC`, 3 filters with `WHERE`, 4 summarises with `AVG()`, 5 uses `GROUP BY` with `COUNT(*)`, and 6 combines sorting with `LIMIT`.

13 Practice Questions and Learning Roadmap

Try these exercises on your own using a new `CollegeDB` database. Write the SQL, run it and check the result against what you expected.

No.	Exercise
1	Create a database named `CollegeDB` and select it with `USE`.
2	Create an `Employees` table with `EmployeeID`, `Name`, `Department` and `Salary`.
3	Insert at least five employee records.
4	Display only the `Name` and `Salary` columns.
5	Use `WHERE` to show employees with a salary above a value you choose.
6	Sort the employees by `Salary` from highest to lowest with `ORDER BY`.
7	Find the average salary.
8	Use `GROUP BY` to find the number of employees in each department.
9	Create a `Departments` table and use an `INNER JOIN` to show each employee with their department name.
10	Use a subquery to list employees earning more than the average salary.

WHAT'S NEXT?

SQL is the foundation for working with data. It retrieves and summarises the data that **analytics** tools study, that **reports** present, and that **business intelligence** dashboards display. Each topic below builds on the skills in this sample.

- Python Programming
- Python for Data Analytics
- NumPy and Pandas
- Data Visualization
- Advanced Excel
- SQL and Database Management
- Power BI and Business Intelligence
- Machine Learning and Artificial Intelligence

CONTINUE YOUR LEARNING JOURNEY

Want to master SQL and other industry-relevant technical skills? Explore our complete educational notes covering SQL, Python, Data Analytics, Advanced Excel, Power BI, and more.

PINAKI IT CONSULTANT PRIVATE LIMITED

www.pinakiithub.com

Final page of the sample learning material | Thank you for reading